# Modern Application Development II - Project Report

**Project:** Trekking Management Application (V2) | **Submitted By:** Aryan Sharma | **Reg No:** 23f3000285

## 1. Project Details & Approach

The problem statement requires the development of a Trekking Management Application (TMA) to help adventure organisations efficiently categorise and manage treks, staff, and trekkers. The core requirment was to build a decoupled application using a specified tech stack involving Flask, Vue.js, SQLite, Redis, and Celery.

My approach was to strictly separate the backend and frontend architectures. The backend is built as a pure monolithic API using Flask, which handles all business logic, role-based access control via JWT tokens, and database interactions through Flask-SQLAlchemy. The frontend is a Single Page Application (SPA) built using Vue.js injected via CDN, ensuring a lightweight enviornment without complex build steps.

For asynchronous operations, I integrated Celery with Redis as the message broker to handle background tasks like monthly HTML reports, daily webhook reminders, and CSV exports. Redis was also utilised to cache frequently accessed API endpoints to optimise response times.

## 2. AI/LLM Declaration

For this project, I utilised AI coding assistants strictly as a learning and debugging tool. Moving from a monolithic Jinja2 application to a decoupled Vue.js frontend presented a learning curve. I used AI to clear my doubts regarding Vue reactivity and to understand the specific configuration syntax required to connect Celery workers with a Redis broker. I did not use AI to blindly generate the core business logic. Every line of code submitted was tested, modified, and is fully understood by me.

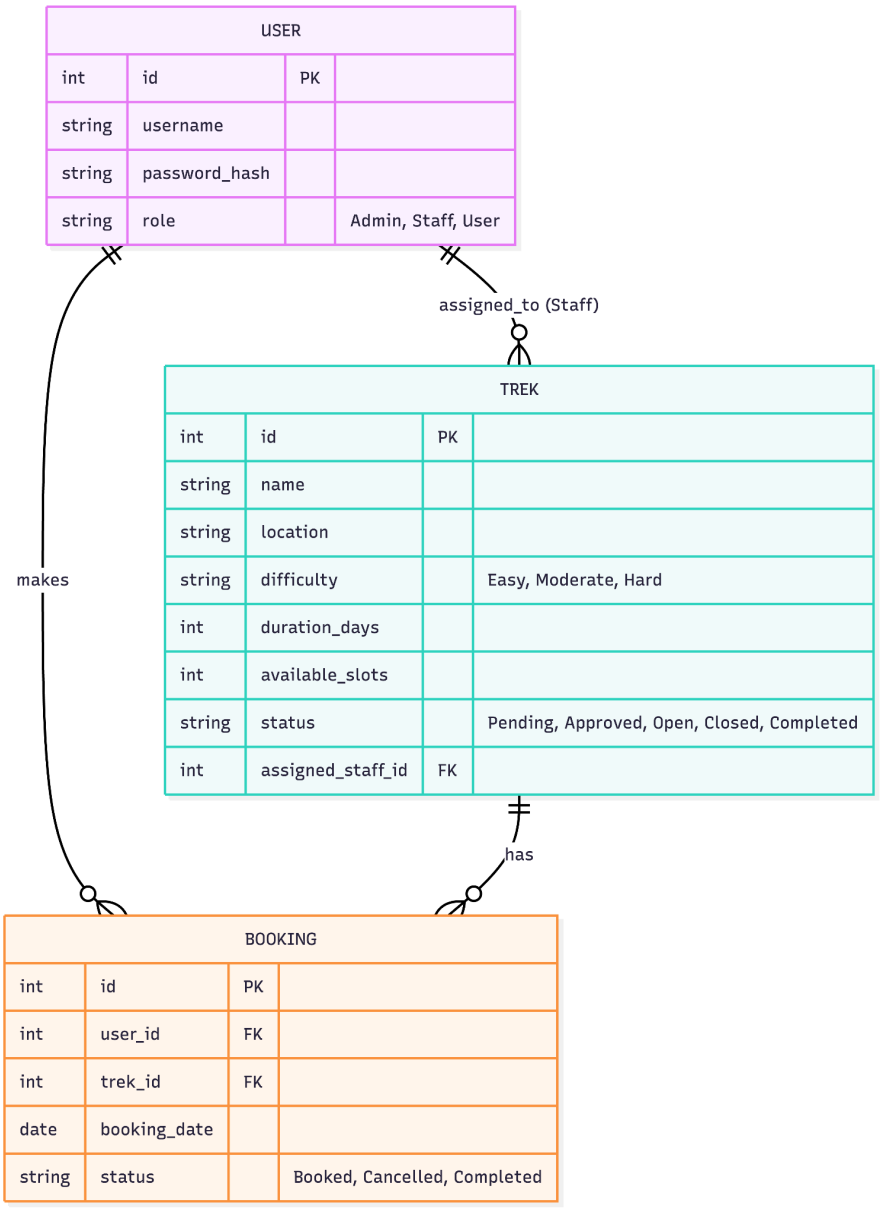
## 3. Frameworks and Libraries Used

| Technology | Purpose in Project |
| --- | --- |
| **Flask (Python)** | Backend framework used to create the JSON API and handle business logic. |

| | |
|---|---|
| **Vue.js (via CDN)** | Frontend framework used to build the dynamic Single Page Application UI. |
| **SQLite & SQLAlchemy** | Programmatic database creation and Object-Relational Mapping. |
| **Flask-JWT-Extended** | Used for secure, token-based authentication and role verification. |
| **Celery & Redis** | Used for asynchronous batch jobs (exports, reminders) and API caching. |
| **Bootstrap 5 (via CDN)** | Sole CSS framework used for responsive HTML generation and styling. |

## 4. ER Diagram & Database Relations

The database was created programmatically and consists of three primary entities linked via foreign keys to manage the trekking system.




- **User Table:** Stores Admin, Trek Staff, and Trekkers. Distinguishes them using a role column.
- **Trek Table:** Stores the trekking routes, duration, difficulty, and capacity. Links to the User table via assigned_staff_id.
- **Booking Table:** Manages trekker reservations. Links to both User and Trek tables via foreign keys.

## 5. API Resource Endpoints

The frontend communicates with the backend exclusively through these primary JSON endpoints:

| Endpoint | Method | Role Access | Description |
|---|---|---|---|
| /api/login | POST | Public | Authenticates user and returns JWT token. |
| /api/admin/treks | POST | Admin | Creates a new trek and assigns staff. |
| /api/staff/treks/<id> | PUT | Staff | Updates available slots and trek status. |
| /api/user/book | POST | User | Processes a new trek booking for a user. |
| /api/export | GET | User | Triggers Celery job to export booking history as CSV. |
| /api/treks | GET | Public | Returns a Redis-cached list of open treks. |

## 6. Presentation Video Link

**Google Drive Link:**
https://drive.google.com/file/d/1F68PjIaOTaov-ZNwakFRR78enktSUYAw/view?usp=sharing